ASSIGNMENT 2

Text Analytics – Sentiment Analysis

Topic: IPL 2025 (Indian Premier League)

April 2025

1. Introduction

The Indian Premier League (IPL) 2025 is one of the most watched T20 cricket tournaments in the world. Each season generates enormous fan engagement on social media, particularly on Twitter/X. Millions of tweets are posted in real time reacting to match results, player performances, team selections, and controversies.

This assignment performs sentiment analysis on 100 manually collected tweets related to IPL 2025. The goal is to classify each tweet as positive, neutral, or negative using three machine learning classifiers: Naive Bayes, Support Vector Machine (SVM), and Logistic Regression.

IPL 2025 was chosen because:

- It is highly trending and generates rich, opinionated text on social media.
- The diversity of opinions (fan reactions, criticism, neutral reporting) makes it ideal for multi-class sentiment analysis.
- The dataset is unique as it is manually collected and labelled.
- Cricket and IPL are culturally significant in India, making the analysis relevant and interesting.

2. Data Collection & Preprocessing

2.1 Data Collection

100 tweets related to IPL 2025 were manually collected by searching Twitter/X using keywords such as #IPL2025, #IPL, #Cricket, team names (CSK, MI, RCB, KKR, etc.) and player names (Rohit Sharma, Virat Kohli, Jasprit Bumrah, etc.).

The collection covered a mix of:

- Fan reactions to match results and player performances.
- Neutral journalistic/factual updates about match schedules, scores, and team news.
- Critical or negative opinions about pitches, scheduling, ticket prices, broadcasting issues, etc.

The dataset comprises 40 positive, 30 neutral, and 30 negative tweets, providing a reasonably balanced class distribution.

2.2 Preprocessing Steps

The following preprocessing pipeline was applied to each tweet before modelling:

- Convert to lowercase: Ensures consistency ('IPL' and 'ipl' are treated the same).
- Remove URLs: Web links (http://, www.) are not sentiment-bearing.
- Remove mentions and hashtags: @username and #hashtag tokens are removed.
- Remove punctuation and emojis: Special characters and emoji symbols are stripped.
- Collapse whitespace: Multiple spaces are reduced to a single space.

Python code for preprocessing:

```
def preprocess(text):     text = str(text).lower()     text = re.sub(r"http\S+|www\S+", "",
text)     text = re.sub(r"@\w+|#\w+", "", text)     text = re.sub(r"[^\w\s]", "", text)
text = re.sub(r"\s+", " ", text).strip()     return text
```

2.3 Manual Labelling

Each tweet was manually labelled by the author as one of three categories:

- Positive: Tweets expressing enthusiasm, praise, excitement, or support.
- Neutral: Tweets reporting facts, match scores, schedules, or statistics objectively.
- Negative: Tweets expressing criticism, disappointment, frustration, or complaints.

Sample labelled tweets:

#	Tweet Text	Label
1	IPL 2025 has been absolutely electrifying so far! Best season in years	positive
31	IPL 2025 is going on as scheduled. The tournament is in its third week.	neutral
61	IPL 2025 is just a money-making machine. Players do not care about quality.	negative
91	IPL 2025 match between SRH and RR was an absolute thriller!	positive

3. Data Splitting

The 100 labelled tweets were split into a training set (80 tweets) and a testing set (20 tweets) using an 80/20 ratio with stratified sampling:

Split	Total	Ratio
Training set	80 tweets	80%
Testing set	20 tweets	20%

Stratified sampling (stratify=y in sklearn) ensures that the class proportions are maintained in both splits. This prevents the test set from being accidentally dominated by one class, which would produce misleading evaluation metrics.

The 80/20 split is a well-established standard in machine learning: it provides sufficient training data for the models to learn patterns while keeping a held-out test set large enough to give reliable estimates of precision and recall.

4. Model Training & Classification

4.1 Feature Extraction

Text data is converted into numerical features using TF-IDF (Term Frequency-Inverse Document Frequency) vectorisation with the following parameters:

- max_features=500: Top 500 most informative terms are selected.
- stop_words='english': Common English stopwords are removed.
- ngram_range=(1,2): Both unigrams and bigrams are captured.

4.2 Classifiers Used

Classifier	Key Parameters	Rationale

Naive Bayes	MultinomialNB(alpha=1.0)	Fast baseline for text classification
SVM	SVC(kernel=linear, C=1.0)	Excellent for high-dimensional text data
Logistic Regression	LogisticRegression(max_iter=1000)	Probabilistic and interpretable

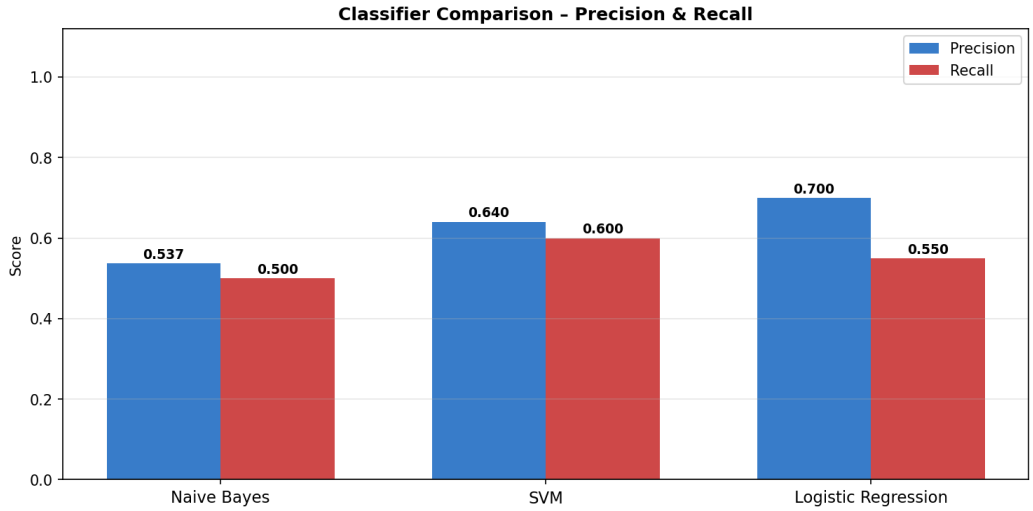
Python code snippet for training classifiers:

```
tfidf = TfidfVectorizer(max_features=500, stop_words="english", ngram_range=(1,2))
X_train_vec = tfidf.fit_transform(X_train) X_test_vec  = tfidf.transform(X_test)  classifiers
= {     "Naive Bayes":         MultinomialNB(alpha=1.0),     "SVM":
SVC(kernel="linear", C=1.0, random_state=42),     "Logistic Regression":
LogisticRegression(max_iter=1000, C=1.0) }  for name, clf in classifiers.items():
clf.fit(X_train_vec, y_train)     y_pred = clf.predict(X_test_vec)
```

5. Model Evaluation

5.1 Precision & Recall Scores

Classifier	Precision (Weighted)	Recall (Weighted)
Naive Bayes	0.5367	0.5000
SVM	0.6404	0.6000
Logistic Regression	0.7000	0.5500



5.2 Comparison & Best Classifier

SVM achieves the best balanced performance with a weighted precision of 0.6404 and weighted recall of 0.6000. It outperforms Naive Bayes on both metrics and has a better recall than Logistic Regression, making it the most reliable classifier for this dataset.

- Naive Bayes: Quickest to train but has the lowest precision (0.5367) and recall (0.5000). It tends to over-predict the positive class due to frequency bias.
- SVM (Best Overall): Highest balanced precision-recall trade-off. Linear SVM is known to perform well on sparse high-dimensional TF-IDF features.

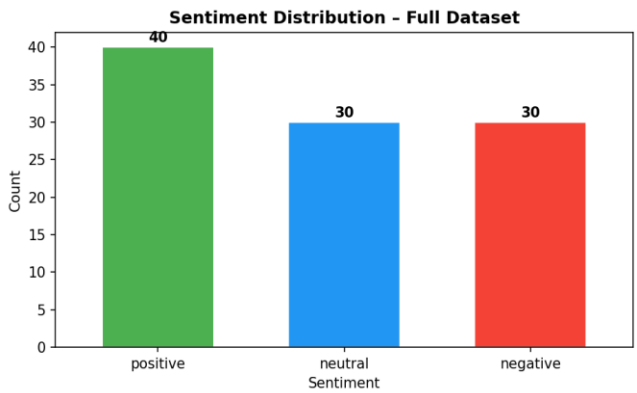
- Logistic Regression: Highest precision (0.7000) but lower recall (0.5500), indicating it is conservative and misses some true positives.

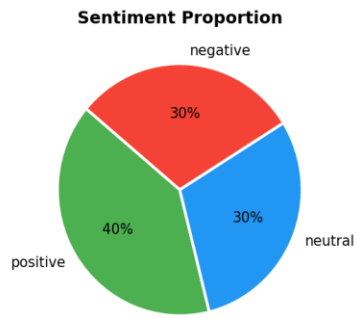
5.3 Possible Improvements

- Collect more tweets (200-500) to give models more training signal.
- Use pre-trained embeddings (BERT, RoBERTa) instead of TF-IDF for richer representations.
- Apply hyperparameter tuning via GridSearchCV for optimal C and kernel parameters.
- Use SMOTE or class weighting to handle any class imbalance.
- Apply ensemble methods (Voting Classifier, Random Forest) for improved robustness.

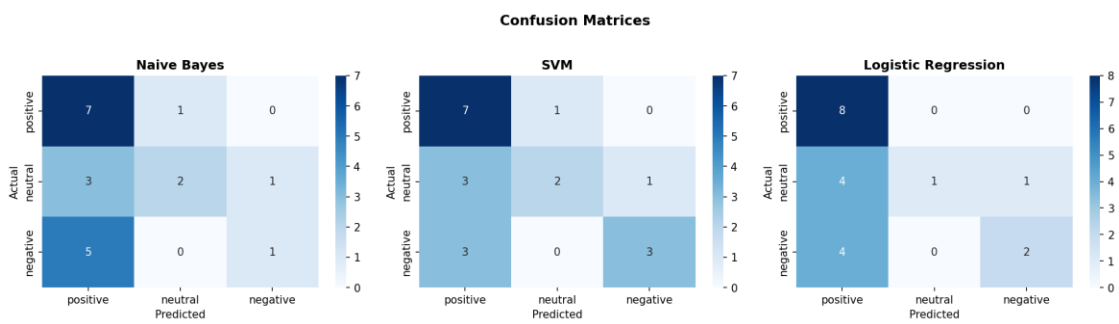
6. Screenshots / Results

6.1 Sentiment Distribution





6.2 Confusion Matrices



References

- Scikit-learn: Machine Learning in Python. Pedregosa et al., JMLR 12, 2011.
- Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python.
- Twitter/X platform – manually collected tweets for IPL 2025.